Project
Breakdown

Create a class “power service”
INPUT – (POSTAL CODE AND DISTRIBUTION HUB) INFORMATION

POSTAL CODE – 6 CHARACTER UNIQUE STRING IDENTIFIER

1. POSTAL CODE IDENTIFIER
2. NUMBER OF PEOPLE LIVING IN THE POSTAL CODE (FROM DATABASE)
3. TOTAL AREA COVERED BY POSTAL CODE

DISTRIBUTION HUB

1. ALPHANUMERIC IDENTIFER
2. CO ORDINATES OF HUB IN (x, y) form where 1 unit= 1 m.
3. Set of postal codes and hubs
    a. One hub can serve multiple postal codes
    b. One postal code can be served by multiple hubs
4. Estimated time needed to repair the current hub.

Create a log of:

1. How many hubs have been fixed
2. The length of time it takes to fix
3. Which employee fixed it?

The goal of the log is to determine whether there are any partial repair hubs that the system should investigate.

Class PowerService:

boolean addPostalCode (String postalCode, int population, int area)

1. State True if the provided postal code is a new addition to our collection.
2. Postal code- an exclusive string
3. Population-based database (per square metre which changes every four years)
4. Area - per square metre

boolean addDistributionHub (String hubIdentifier, Point location, Set servicedAreas)

1. It is true if the hub is a new addition to our system.
2. INSTALL a new distribution hub into our system.
3. Hub Universal Transverse Mercator coordinates (point out location)
4. Serviced region denotes the postal code served by the hub.

Void hubDamage (String hubIdentifier, float repairEstimate)

1. New hub identifier
2. Get the hub identifier and store as damaged hub
3. Estimated Time taken to repaire. (in hours)

Void hubRepair(String hubIdentifier, String employeeId, float repairTime, boolean inService)
1. State that the EMPLOYEE assigned has completed the work in the right manner. Time for a specific hub.
2. Condition
   a. If the hub is fixed, set inService to True.
   b. If the hub is not repaired, set it to false so that the employee may use hubDamage() again.

According to company procedure, the employee would report this repair using their mobile phone app before leaving the power distribution centre.

int peopleOutOfService ()
1. Determine the number of people who are no longer in service.
2. Count all repair requests and hub damage that are deemed out of service until the employee updates the hub is fully fixed.
3. The fraction equals 1/ (number of hubs serving the postal codes)

List mostDamagedPostalCodes (int limit)
1. Calculate limit
2. List out the postal codes in descending order in comparison of repairTime

List fixOrder (int limit)
1. The significance of a hub to fix is the number of people who regain service per hour of repair.

List rateOfServiceRestoration (float increment)
1. Estimate rate of time to restore power
2. Calculate the less transit time to move between two hubs where hubs are in descending order of damage.
3. Return List of hours of repair effort.
4. The number of hours of system-wide repairs necessary until (x * increment) percent of the province's population has access to power is reported in list entry x. For instance, if the increment is 0.05 (i.e., 5%), entry 0 would be the repair time to restore power to 0% of the population (and would likely take 0 hours), entry 1 would be the repair time to restore power to 5% of the population, entry 2 would be the repair time to restore power to 10% of the population, and so on.

List repairPlan (String startHub, int maxDistance, float maxTime)
1. The route provided by the returned value has to meet the requirements below. The direction of the journey from startHub to endHub must be monotonic, either in x or y. The required repair time at the intermediate hubs is limited to maxTime. The set of

intermediate hubs treated as a group must have the greatest influence on being fixed throughout all conceivable pathways.

2. The employee will start at the given “startHub” location. That location should be the first location in the returned list. The last location in the returned list should be the hub that is within “maxDistance” metres of the start hub and that has the largest repair impact among all hubs within that “maxDistance” range. The idea is that the employee will start by fixing startHub and will then want to travel to that next impactful hub to repair it. For the rest of this description we will call this next impactful hub the “endHub”. As the employee drives from startHub to the endHub, they will pass other hubs that they could fix along the way. The list of the hubs along the way that they should fix are given, in order of fixing, by the returned list.
3. Because there are several hubs allocated to a single postal code, we must create a map with the key as the postal code and the list of values as the hubs.

List underservedPostalByPopulation (int limit) & List underservedPostalByArea (int limit)

1. Still confused about the hierarchy of postal codes given.
2. If the average population serves postal codes A and B, postal code A is less serviced than postal code B.
3. The number of hubs per person in A is less than that in B. If Postal Code A is less served than Postal Code B in terms of area,
4. In the postal code, A has fewer hubs per square metre than B.

Support classes:

1. **Class Point**:
   a. Container for 2d (x, y) hub coordinate point (include getters and setters)
2. **Class HubImpact:**
   b. getHubID AND getImpact) and setters
3. **Class DamagedPostalCodes:**
   c. (getPostalcode and getRepairEstimate) and setters

Example Explained:

1. We take into account the influence of all hubs inside that circle.
2. We've determined that hub B has the most influence, therefore we'll take a path from A to B. Because they are within the rectangle described by A and B, only hubs E, J, and K are considered stopping places along that trip. The second figure shows how trails might appear.
3. However, it fails on the property of crossing the diagonal from A to B twice, assuming that K is above the diagonal and both E and J are below.
4. Similarly, the y-monotone route only increases the Y value, hence it fulfils the monotone condition.
5. The third path, which is not monotone, has the x coordinate on the path rise (A -> J) and drop (J -> K), indicating that it is not x-monotone.